

SmartRACK

ePDU Inteligente

DOCUMENTACIÓN TÉCNICA

Versión 1.0 — Agosto 2026

Campo	Detalle
Proyecto	SmartRACK — ePDU Inteligente
Cliente	AucaTek
Institución	Instituto Leonardo Murialdo
Carrera	7mo Informática A
Año	2026
Versión del documento	1.0
Estado	En desarrollo

SmartRACK

ePDU Inteligente

DOCUMENTACIÓN TÉCNICA

Sección 2 — Introducción

Campo	Detalle
Proyecto	SmartRACK — ePDU Inteligente
Cliente	AucaTek
Institución	Instituto Leonardo Murialdo
Carrera	7mo Informática A
Año	2026
Versión del documento	1.0
Estado	En desarrollo

1. Control de Cambios

La siguiente tabla registra las modificaciones realizadas sobre este documento a lo largo del desarrollo del proyecto SmartRACK.

Versión	Fecha	Responsable	Descripción del cambio
1.0	22/08/2026	Ezequiel Morgado	Creación del documento. Redacción de la Sección 2 — Introducción completa.
—	—	—	—

2. Introducción

2.1 Propósito

El presente documento tiene como propósito describir de manera formal y detallada el sistema SmartRACK, una Unidad de Distribución de Energía Electrónica (ePDU) inteligente desarrollada para AucaTek. SmartRACK resuelve una necesidad concreta en entornos de infraestructura tecnológica: la imposibilidad de monitorear y controlar remotamente las tomas de corriente de un rack de servidores sin intervención física en el lugar.

Los administradores de datacenters y salas de servidores enfrentan problemas recurrentes como la incapacidad de detectar consumos anómalos en tiempo real, la necesidad de desplazarse físicamente para apagar o encender equipos conectados al rack, y la falta de registros históricos de consumo eléctrico. SmartRACK aborda estos problemas integrando hardware de medición eléctrica con un portal web accesible desde cualquier dispositivo con conexión a internet.

Este documento está dirigido al equipo de desarrollo del proyecto, al cliente (AucaTek) y a los evaluadores/profesores del Instituto Leonardo Murialdo. Sirve como referencia técnica durante el desarrollo y como evidencia formal de las decisiones de diseño tomadas.

2.2 Alcance

SmartRACK en su versión 1 (MVP) comprende los siguientes componentes y funcionalidades:

Componente / Funcionalidad	Incluido en v1	Previsto para v2
Control ON/OFF de 4 tomas inteligentes	Sí	Sí
Monitoreo eléctrico en tiempo real (V, A, W, PF, Hz, kWh)	Sí (Premium)	Sí (Premium)
Monitoreo de temperatura y humedad interna	Sí (Premium)	Sí (Premium)
Alertas por umbral con notificación por correo	No	Sí
Portal web con 3 roles (Admin, Operator, Viewer)	Sí	Sí
Exportación de datos históricos en CSV	Sí (Premium)	Sí (Premium)
Sistema de licencias Normal/Premium	Sí	Sí
PCB fabricada y gabinete industrial	Sí	Sí (Mejorado)
OTA (actualización de firmware inalámbrica)	No	Sí
Display OLED en el dispositivo	No	Sí
Broker MQTT propio (Mosquitto en servidor Claro)	No	Sí
Soporte completo para múltiples PDUs simultáneos	Parcial	Completo

2.3 Personal involucrado

El siguiente equipo participó en el diseño, desarrollo e implementación del sistema SmarTRACK:

Nombre	Rol	Área de responsabilidad
Ezequiel Morgado	Project Manager Backend Developer	Base de datos, API, MQTT, ciberseguridad, integración general, control y gestión completa del proyecto
Camilo Spinelli	Dev Firmware	Firmware ESP32 y sensores
Sofía Hermosilla	Dev Frontend / UX UI	Diseño de interfaz, dashboards, experiencia de usuario
Violeta Martini	Diseño Industrial	Diseño de los sensores, piezas y gabinete PDU en AutoCAD
Martín Barral	Asesor Técnico	Revisión del diseño electrónico y PCB, guías orientadas a la realización eficiente del proyecto
Diego Martini	CEO AucaTek / Cliente	Definición de requerimientos, validación técnica, firma del contrato

2.4 Definiciones, acrónimos y abreviaturas

Término / Acrónimo	Definición
ePDU	Electronic Power Distribution Unit. Unidad de distribución de energía electrónica con capacidad de monitoreo y control remoto.
PDU	Power Distribution Unit. Dispositivo que distribuye energía eléctrica a múltiples equipos en un rack.
MQTT	Message Queuing Telemetry Transport. Protocolo de mensajería liviano basado en publicación/suscripción, diseñado para IoT.
TLS	Transport Layer Security. Protocolo criptográfico que garantiza la comunicación segura a través de redes.
PZEM	Módulo de medición de energía eléctrica AC fabricado por Peacefair. Mide voltaje, corriente, potencia, factor de potencia, frecuencia y energía acumulada.
AHT10	Sensor digital de temperatura y humedad fabricado por AOSONG Electronics. Comunicación I2C.
ESP32	Microcontrolador de doble núcleo fabricado por Espressif Systems. En este proyecto se usa exclusivamente con Ethernet (sin WiFi).
W5500	Controlador Ethernet hardwired fabricado por WIZnet. Implementa el stack TCP/IP en hardware, conectado al ESP32 mediante SPI.

Término / Acrónimo	Definición
UART	Universal Asynchronous Receiver-Transmitter. Protocolo de comunicación serie asíncrono. Usado por el PZEM-004T.
I2C	Inter-Integrated Circuit. Protocolo de comunicación serie síncrono de dos hilos (SDA y SCL). Usado por el AHT10.
SPI	Serial Peripheral Interface. Protocolo de comunicación serie síncrono de cuatro hilos. Usado por el W5500.
DHCP	Dynamic Host Configuration Protocol. Protocolo que asigna automáticamente una dirección IP al dispositivo en la red local.
NTP	Network Time Protocol. Protocolo de sincronización de hora a través de internet.
MVP	Minimum Viable Product. Versión mínima funcional del producto con las características esenciales.
PHP	Hypertext Preprocessor. Lenguaje de programación del lado del servidor usado en el portal web.
CSRF	Cross-Site Request Forgery. Ataque de seguridad web que fuerza acciones no deseadas en nombre de un usuario autenticado. SmarTRACK implementa tokens CSRF para prevenirlo.
CSP	Content Security Policy. Header HTTP de seguridad que controla los recursos que el navegador puede cargar.
HSTS	HTTP Strict Transport Security. Header HTTP que obliga al navegador a usar HTTPS.
MCB	Miniature Circuit Breaker. Disyuntor termomagnético de protección de circuito eléctrico.
MOV	Metal Oxide Varistor. Componente electrónico de protección contra picos de tensión.
LWT	Last Will and Testament. Mensaje MQTT que el broker publica automáticamente cuando un cliente se desconecta abruptamente.
QoS	Quality of Service. Nivel de garantía de entrega de un mensaje MQTT (0: sin garantía, 1: al menos una vez, 2: exactamente una vez).
IRAM	Instituto Argentino de Normalización y Certificación. Define las normas de enchufes y tomacorrientes vigentes en Argentina.

2.5 Referencias

Las siguientes fuentes fueron consultadas durante el desarrollo del proyecto SmartRACK. El formato de citación utilizado corresponde a la 7ma edición de las normas APA (American Psychological Association).

- Espressif Systems. (2024). ESP32 Series Datasheet v5.3. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Espressif Systems. (2024). ESP32 Technical Reference Manual v5.1. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- Peacefair. (2019). PZEM-004T V3.0 User Manual — AC Communication Module. <https://innovatorsguru.com/wp-content/uploads/2019/06/PZEM-004T-V3.0-Datasheet-User-Manual.pdf>
- AOSONG Electronics. (2020). AHT10 Technical Manual — Temperature and Humidity Sensor. https://server4.eca.ir/eshop/AHT10/Aosong_AHT10_en_draft_0c.pdf
- WIZnet. (2013). W5500 Datasheet v1.1.0 — Hardwired TCP/IP Embedded Ethernet Controller. https://docs.wiznet.io/img/products/w5500/W5500_ds_v110e.pdf
- HiveMQ. (2026). MQTT Essentials: A ten-part series on the MQTT protocol. <https://www.hivemq.com/blog/mqtt-essentials-part-1-introducing-mqtt/>
- HiveMQ. (2026). HiveMQ Cloud Documentation — Getting Started. <https://docs.hivemq.com/hivemq-cloud/latest/>
- OPeN Lab — Oregon State University. (2022). SSLClient Library for Arduino. GitHub. <https://github.com/OPeNLab-OSU/SSLClient>
- Knolleary. (2023). PubSubClient — MQTT library for Arduino. GitHub. <https://github.com/knolleary/pubsubclient>
- Tabler. (2023). Tabler — Premium and Open Source Dashboard Template. <https://tabler.io/>
- Chart.js Contributors. (2023). Chart.js — Simple yet flexible JavaScript charting library. <https://www.chartjs.org/>
- The PHP Group. (2024). PHP 8.2 Manual. <https://www.php.net/manual/es/>
- alwaysdata. (2024). Documentación de la plataforma alwaysdata. <https://help.alwaysdata.com/>
- OASIS Open. (2019). MQTT Version 5.0 — OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

2.6 Resumen ejecutivo

SmartRACK es un sistema de distribución de energía inteligente (ePDU) desarrollado por el equipo de séptimo año de Informática del Instituto Leonardo Murialdo, en colaboración con AucaTek como empresa cliente. El proyecto surge de la necesidad identificada por AucaTek de ofrecer a sus clientes un producto de monitoreo y control de racks que sea accesible, confiable y desarrollado íntegramente en Argentina.

El sistema está compuesto por dos partes principales. La primera es el hardware: un dispositivo basado en ESP32 con módulo Ethernet W5500 que se conecta directamente al rack mediante un cable de red estándar. El hardware integra un sensor PZEM-004T para medición eléctrica (voltaje, corriente, potencia, factor de potencia, frecuencia y energía acumulada) y un sensor AHT10 para temperatura y humedad interna. Cuatro relés permiten el control ON/OFF individual de cada toma del rack.

La segunda parte es el software: un portal web desarrollado en PHP 8.2 con base de datos MySQL, deployado en producción sobre alwaysdata con HTTPS activo. El portal implementa tres niveles de acceso — Administrador, Operador y Viewer — cada uno con permisos diferenciados. La comunicación entre el hardware y el backend se realiza mediante el protocolo MQTT sobre TLS (puerto 8883) a través de un broker HiveMQ Cloud, garantizando la seguridad y confiabilidad del transporte de datos.

SmartRACK incorpora un sistema de licencias que diferencia entre el modo Normal (control de tomas únicamente) y el modo Premium (telemetría completa, gráficos históricos, exportación de datos y alertas automáticas por correo electrónico). La ciberseguridad del portal fue auditada e incluye rate limiting, protección CSRF, headers HTTP de seguridad (CSP, HSTS, X-Frame-Options), registro de eventos de seguridad y validación exhaustiva de entradas.

Público objetivo: Administradores de datacenters, técnicos de infraestructura IT y empresas con salas de servidores que requieran monitoreo remoto de consumo eléctrico y control de equipos sin necesidad de desplazamiento físico.

3. Descripción General

3.1 Perspectiva del producto

SmartRACK es un producto nuevo desarrollado íntegramente por el equipo del proyecto en colaboración con AucaTek. No es una mejora ni una extensión de ningún sistema preexistente dentro de la empresa. AucaTek no contaba previamente con ningún producto de este tipo en su empresa, por lo que SmartRACK representa su primera incursión en el segmento de hardware inteligente para infraestructura IT.

El sistema opera de forma autónoma y no depende de ninguna plataforma de terceros para su funcionamiento básico: el hardware ESP32 puede controlar las tomas y servir la interfaz web local incluso sin conexión a internet. La conectividad con el portal web en la nube es una funcionalidad adicional que amplía las capacidades del sistema, no un requisito para su operación mínima.

Desde el punto de vista arquitectónico, SmartRACK es un sistema embebido conectado a internet (IoT) que combina un dispositivo físico (hardware + firmware) con un portal web en la nube (backend + frontend). La comunicación entre ambas partes se realiza exclusivamente mediante el protocolo MQTT sobre TLS, garantizando la confidencialidad y autenticidad de los datos transmitidos.

3.2 Funcionalidad del producto

SmartRACK provee las siguientes funcionalidades principales:

Funcionalidad	Descripción
Control de tomas	Encendido y apagado individual de hasta 4 tomas de 220V AC mediante relés controlados por el ESP32. El control puede realizarse desde la interfaz web local (red LAN) o desde el portal web en la nube (internet).
Medición eléctrica	El sensor PZEM-004T mide en tiempo real voltaje (V), corriente (A), potencia activa (W), factor de potencia, frecuencia (Hz) y energía acumulada (kWh). Los datos se publican cada 30 segundos mediante MQTT y se almacenan en la base de datos (modo Premium).
Monitoreo ambiental	El sensor AHT10 mide temperatura y humedad interna del PDU cada 60 segundos. Los datos se publican y almacenan de la misma forma que la telemetría eléctrica (modo Premium).
Alertas automáticas	El firmware detecta condiciones fuera de umbral (voltaje < 210V, voltaje > 240V, temperatura > 45°C) y publica alertas vía MQTT. El backend las procesa, las registra y envía notificaciones por correo electrónico al administrador.
Portal web	Interfaz web responsive accesible desde cualquier navegador, con autenticación por usuario y contraseña. Implementa tres roles: Administrador (control total), Operador (control de tomas y visualización) y Viewer (solo lectura).
Sistema de licencias	El sistema opera en modo Normal (control de tomas únicamente) o modo Premium (telemetría completa, gráficos históricos, exportación CSV y alertas). La activación Premium se realiza mediante códigos de licencia administrados por AucaTek.
Interfaz web local	El ESP32 sirve una interfaz HTTP en la red local (puerto 80) que permite controlar las tomas directamente desde la LAN, sin necesidad de internet ni del portal web en la nube.

Funcionalidad	Descripción
Buffer offline	Si se pierde la conexión a internet, el firmware almacena las lecturas en memoria. Al recuperar la conexión, las publica automáticamente al broker con la marca is_buffered=1.
Exportación de datos	Los usuarios con modo Premium pueden exportar la telemetría histórica en formato CSV, con filtros por rango de fechas (modo Premium).
Gestión de usuarios	El administrador puede crear, editar y eliminar usuarios, asignarles roles y vincularlos a un PDU específico. Requiere verificación de identidad (contraseña) para proteger operaciones sensibles.

3.3 Características de los usuarios

A continuación se describen los perfiles de usuario del sistema SmartRACK, siguiendo el formato de caracterización de usuarios establecido en las Buenas Prácticas en la Dirección y Gestión de Proyectos Informáticos (UTN).

Tipo de usuario	Administrador
Formación	Técnico en sistemas, administrador de infraestructura IT o responsable técnico de la empresa. Conocimientos en redes, servidores y administración de sistemas. Manejo fluido de interfaces web.
Actividades	Gestión completa del portal: crear y administrar usuarios, gestionar PDUs, visualizar telemetría en tiempo real, configurar alertas, exportar datos históricos, activar licencias Premium y controlar individualmente cada toma del rack.

Tipo de usuario	Operador
Formación	Técnico IT o personal de soporte con conocimientos básicos de infraestructura. No requiere conocimientos de administración de sistemas avanzados.
Actividades	Control ON/OFF de tomas, visualización de telemetría en tiempo real, consulta de alertas y logs del sistema. No puede gestionar usuarios ni PDUs.

Tipo de usuario	Viewer (Solo Lectura)
Formación	Personal de supervisión, gerencia o auditores que necesitan visibilidad del estado del rack sin capacidad de modificación. No requiere conocimientos técnicos específicos.
Actividades	Visualización del estado de las tomas, lectura de métricas eléctricas y ambientales en tiempo real, consulta de alertas y logs. No puede controlar tomas ni acceder a configuraciones.

Tipo de usuario	Cliente AucaTek (externo)
Formación	Empresa o profesional que adquiere el producto SmartRACK. Puede ser administrador de datacenter, proveedor de servicios IT o empresa con sala de servidores propia. Nivel técnico variable.
Actividades	Uso del portal web como usuario final. Gestión de su propia instalación de SmartRACK. Activación y renovación de licencias Premium. Contacto con AucaTek para soporte técnico.

3.4 Restricciones

Esta sección describe las condiciones, limitaciones y políticas que restringen las opciones disponibles para el desarrollo de SmartRACK.

- **Políticas regulatorias**

Las tomas de salida del PDU deben cumplir con la norma IRAM 2073 vigente en Argentina (enchufes y tomacorrientes de uso domiciliario e industrial). El dispositivo opera a 220V AC y debe respetar las normativas eléctricas nacionales aplicables a equipos de baja tensión.

El software del portal web es propietario de AucaTek. El firmware del ESP32 fue desarrollado íntegramente por el equipo del proyecto y utiliza librerías de código abierto con licencias compatibles con uso comercial (MIT, Apache 2.0). No se utilizan librerías con licencia GPL en el firmware para evitar restricciones de distribución.

- **Limitaciones de hardware**

El ESP32 NodeMCU 38 pines dispone de 520KB de SRAM y 4MB de memoria flash. El buffer offline en memoria está limitado a 20 lecturas PZEM y 10 lecturas AHT10 simultáneas para no comprometer la operación del sistema.

El módulo W5500 (USR-ES1) proporciona conectividad Ethernet 10/100Mbps. No se utiliza el radio WiFi del ESP32 en ningún momento — toda la comunicación de red se realiza exclusivamente por Ethernet. El módulo ocupa el bus SPI del ESP32, compartido con el chip CS en GPIO5.

El PZEM-004T v3 mide hasta 100A con transformador externo. En la versión 1 del proyecto se utiliza el modelo de hasta 10A con shunt interno, suficiente para las cargas de un rack de laboratorio. El AHT10 opera entre -40°C y +80°C con precisión de $\pm 0.3^{\circ}\text{C}$ en temperatura y $\pm 2\%$ en humedad relativa.

- **Interfaces con otras aplicaciones**

SmartRACK interactúa con los siguientes servicios externos:

Servicio	Tipo de interfaz	Uso
HiveMQ Cloud	MQTT sobre TLS (puerto 8883)	Broker de mensajería entre firmware y backend
alwaysdata	SFTP / HTTP(S)	Hosting del portal web y base de datos MySQL
PHPMailer + Gmail SMTP	SMTP (puerto 587 TLS)	Envío de notificaciones por correo electrónico
Google Fonts (CDN)	HTTPS	Fuente tipográfica Montserrat en el portal web
jsDelivr / cdnjs (CDN)	HTTPS	Librerías Tabler.io, Font Awesome y Chart.js
pool.ntp.org	UDP (puerto 123)	Sincronización horaria del firmware (NTP)

- **Función de control**

El acceso al portal web requiere autenticación con usuario y contraseña. Las contraseñas se almacenan hashadas con bcrypt en la base de datos. Se implementa rate limiting: tras 5 intentos fallidos en una ventana de 10 minutos, la IP queda bloqueada por 15 minutos.

Cada sesión autenticada tiene un rol asociado (admin, operator, viewer) que determina las acciones permitidas. La verificación de rol se realiza en el servidor en cada request — no es posible escalar privilegios desde el cliente. Las operaciones sensibles (gestión de usuarios) requieren verificación adicional de identidad mediante contraseña.

- **Requisitos del lenguaje**

Toda la interfaz del portal web, los mensajes del sistema, las alertas, los correos de notificación y la documentación están en español (variante Argentina). El firmware incluye mensajes de diagnóstico en español en el puerto serie para facilitar la depuración por parte del equipo de hardware.

- **Protocolos señalados**

Protocolo	Capa	Uso en SmartRACK
HTTP/HTTPS	Aplicación	Portal web del cliente. HTTPS obligatorio en producción (alwaysdata).
MQTT v3.1.1	Aplicación	Comunicación bidireccional entre firmware y backend vía HiveMQ Cloud.
TLS 1.2/1.3	Transporte seguro	Cifrado de la conexión MQTT (puerto 8883) mediante SSLClient + BearSSL.
UART (RS-485)	Enlace de datos	Comunicación entre ESP32 y PZEM-004T via Serial2 (GPIO16/17).
I2C	Enlace de datos	Comunicación entre ESP32 y AHT10 (SDA=GPIO21, SCL=GPIO22).
SPI	Enlace de datos	Comunicación entre ESP32 y W5500 (VSPI: SCK=18, MISO=19, MOSI=23, CS=5).
TCP/IP	Red / Transporte	Stack TCP/IP implementado en hardware por el chip W5500.
DHCP	Red	Asignación automática de IP al ESP32 en la red local.
NTP (UDP)	Aplicación	Sincronización horaria del firmware contra pool.ntp.org.

- **Requisitos de fiabilidad**

El portal web en producción (alwaysdata) tiene un uptime garantizado por el proveedor. El firmware del ESP32 implementa reconexión automática al broker MQTT con reintentos cada 5 segundos ante desconexiones. El buffer offline garantiza que no se pierdan lecturas de telemetría durante interrupciones de red de hasta 10 minutos (20 lecturas PZEM a 30 segundos cada una).

Los relés se inicializan en estado OFF al arrancar el firmware, independientemente del estado anterior, para evitar encendidos no deseados tras un reinicio del dispositivo. El sistema tolera la pérdida de internet manteniendo el control local de tomas mediante la interfaz web embebida en el ESP32.

- **Credibilidad**

SmartRACK fue desarrollado en el marco de un proyecto interdisciplinario de cuarto año del Instituto Leonardo Murialdo, con AucaTek como cliente real. El portal web en producción opera sobre HTTPS con certificado

válido. La comunicación MQTT usa TLS con certificados del broker HiveMQ Cloud. El código fuente está versionado en GitHub y el historial de commits es auditable por cualquier evaluador.

AucaTek avala el producto como parte de su línea de soluciones para infraestructura IT. Diego Martini (CEO de AucaTek) participó activamente en la definición de requerimientos y validación de decisiones técnicas a lo largo del desarrollo.

- [Consideraciones de seguridad](#)

El portal web implementa las siguientes medidas de seguridad:

Medida	Implementación
Autenticación	Usuario y contraseña con bcrypt. Cambio obligatorio en el primer login.
Rate limiting	Máximo 5 intentos fallidos en 10 minutos. Bloqueo de IP por 15 minutos almacenado en DB.
Protección CSRF	Token CSRF en todos los formularios y endpoints que modifican datos.
Headers HTTP de seguridad	CSP, HSTS (preload), X-Frame-Options: DENY, X-Content-Type-Options, Referrer-Policy.
Validación de entradas	mysqli_real_escape_string() en todas las queries. htmlspecialchars() en toda salida HTML. Validación is_string() en endpoints JSON.
Registro de seguridad	Tabla security_logs con eventos: login_exitoso, login_fallido, bloqueo_ip, csrf_invalido, acceso_denegado.
Roles y permisos	Verificación de rol en servidor en cada request. Endpoints sensibles validan el rol antes de ejecutar.
Comunicación cifrada	HTTPS en el portal web. MQTT sobre TLS 1.2/1.3 en la comunicación firmware-backend.

3.5 Suposiciones y dependencias

El funcionamiento de SmartRACK asume las siguientes condiciones:

Suposición / Dependencia	Detalle	Riesgo si no se cumple
Red local con DHCP disponible	El ESP32 obtiene su IP automáticamente mediante DHCP al conectarse a la red del cliente.	El dispositivo no obtiene IP y no puede comunicarse. Solución: configurar IP estática en el firmware.
HiveMQ Cloud free tier disponible	El broker MQTT opera bajo el plan gratuito de HiveMQ Cloud, con límite de 100 conexiones simultáneas y 10GB de tráfico mensual.	Si se superan los límites, el broker puede rechazar conexiones. Solución v2: migrar a Mosquitto propio.
alwaysdata disponible	El portal web y la base de datos operan sobre alwaysdata. El plan actual es gratuito con recursos compartidos.	Si el servicio tiene una interrupción, el portal no estará disponible. El hardware sigue funcionando localmente.
Certificado TLS válido en trust_anchors.h	El firmware incluye el certificado leaf de HiveMQ Cloud. El certificado actual vence el 14/09/2026.	Si vence y no se renueva, el handshake TLS falla y el firmware no puede conectarse al broker MQTT.
Cable de red disponible en la ubicación del rack	El ESP32 se conecta por Ethernet. No usa WiFi.	Sin cable de red no hay conectividad. No existe fallback WiFi en v1.
Suministro eléctrico estable de 5V para el ESP32	La fuente switching provee 5V DC al ESP32 y sensores. Se evalúa una fuente de 2A a 4A.	Si la fuente es insuficiente bajo carga máxima (4 relés activos simultáneamente), el ESP32 puede reiniciarse.

3.6 Evolución previsible del sistema

La versión 1 (MVP) de SmartRACK está diseñada teniendo en cuenta las siguientes evoluciones previstas para la versión 2:

Área	Evolución prevista en v2
Hardware	PCB fabricada con chip ESP32 soldado directamente (sin módulo NodeMCU). Diseño de Violeta Martini y Martin Barral en Fusion 360 y AutoCAD para el gabinete industrial.
Firmware	Actualización OTA (Over The Air) sin necesidad de cable USB. Display OLED integrado para visualización local de métricas sin necesidad de navegador.
Conectividad	Migración del broker MQTT de HiveMQ Cloud (gratuito) a una instancia de Mosquitto propia en servidor Claro, eliminando los límites del plan gratuito.
Escalabilidad	Soporte completo para múltiples PDUs simultáneos con un único portal web. Retención de datos históricos con agregación por hora/día para reducir el volumen en base de datos.
Seguridad firmware	Cifrado de credenciales MQTT en memoria flash del ESP32 mediante partición NVS cifrada.
Backend	API REST pública documentada con OpenAPI para integración con sistemas de terceros (Nagios, Zabbix, etc.).
Modelo de negocio	Diego Martini gestiona la distribución y activación de licencias Premium externamente. La v2 incorporará un panel de administración para AucaTek y sus directivos..

Fin de la Sección 3 — Descripción General

4. Requisitos Específicos

4.1 Requerimientos Funcionales

Los siguientes requisitos describen las funcionalidades que el sistema SmartRACK debe proveer. Cada requisito está identificado por un número único, incluye una descripción detallada y una prioridad de implementación.

Nro. de requisito	1	Tipo	Funcional
Requisito	El sistema deberá permitir encender y apagar individualmente cada toma de corriente del PDU.		
Descripción	El administrador y el operador pueden controlar el estado (ON/OFF) de cada una de las 4 tomas inteligentes de forma independiente, desde el portal web o desde la interfaz local del ESP32. El cambio de estado se ejecuta mediante un relé físico y se confirma mediante un mensaje MQTT de retorno del firmware.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	2	Tipo	Funcional
Requisito	El sistema deberá publicar telemetría eléctrica en tiempo real cada 30 segundos.		
Descripción	El firmware lee del sensor PZEM-004T los valores de voltaje (V), corriente (A), potencia activa (W), factor de potencia, frecuencia (Hz) y energía acumulada (kWh), y los publica vía MQTT. El backend los recibe y almacena en la tabla telemetry_pzem. Disponible únicamente en modo Premium.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	3	Tipo	Funcional
Requisito	El sistema deberá publicar telemetría de temperatura y humedad cada 60 segundos.		
Descripción	El firmware lee del sensor AHT10 los valores de temperatura (°C) y humedad relativa (%), y los publica vía MQTT. El backend los recibe y almacena en la tabla telemetry_aht10. Disponible únicamente en modo Premium.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	4	Tipo	Funcional
Requisito	El sistema deberá detectar condiciones fuera de umbral y generar alertas automáticas.		
Descripción	El firmware detecta: voltaje < 210V (voltaje_bajo), voltaje > 240V (voltaje_alto) y temperatura > 45°C (temperatura_alta). Publica un JSON en el tópico pdu/{codigo}/alertas. El backend lo registra en la tabla alertas y envía notificación por correo. Se evita duplicación verificando si ya existe una alerta activa del mismo tipo.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	5	Tipo	Funcional
Requisito	El sistema deberá implementar autenticación con tres roles diferenciados.		
Descripción	El portal requiere login con usuario y contraseña hasheada con bcrypt. Los roles son: Administrador (control total), Operador (control de tomas y visualización) y Viewer (solo lectura). El primer login obliga a cambiar la contraseña por defecto.		
Prioridad	Alta/Esencial [X] Media/Deseado [] Baja/Opcional []		

Nro. de requisito	6	Tipo	Funcional
Requisito	El sistema deberá permitir la gestión de usuarios (alta, baja, modificación y consulta).		
Descripción	El administrador puede crear, editar y eliminar usuarios asignándoles rol, contraseña y PDU vinculado. La operación requiere verificación adicional de identidad mediante contraseña. Disponible únicamente para el rol Administrador.		
Prioridad	Alta/Esencial [X] Media/Deseado [] Baja/Opcional []		

Nro. de requisito	7	Tipo	Funcional
Requisito	El sistema deberá permitir la gestión de PDUs (alta, baja, modificación y consulta).		
Descripción	El administrador puede registrar nuevos PDUs, editarlos y desactivarlos. Cada PDU tiene un codigo_pdu derivado de la MAC del ESP32, IP local, modo (normal/premium) y estado de conexión basado en el campo ultimo_contacto.		
Prioridad	Alta/Esencial [X] Media/Deseado [] Baja/Opcional []		

Nro. de requisito	8	Tipo	Funcional
Requisito	El sistema deberá implementar un sistema de licencias Normal y Premium.		
Descripción	El modo Normal permite únicamente el control de tomas. El modo Premium habilita telemetría, gráficos, exportación CSV y alertas. La activación se realiza mediante un código de licencia ingresado por el administrador. El sistema registra la fecha de vencimiento y envía avisos de gracia.		
Prioridad	Alta/Esencial [X] Media/Deseado [] Baja/Opcional []		

Nro. de requisito	9	Tipo	Funcional
Requisito	El sistema deberá almacenar lecturas en un buffer offline cuando no haya conexión MQTT.		
Descripción	Si el firmware pierde la conexión al broker, las lecturas se guardan en buffers circulares en memoria (máximo 20 lecturas PZEM y 10 AHT10). Al recuperar la conexión, se publican automáticamente con la marca is_buffered=1.		

Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐
-----------	---

Nro. de requisito	10	Tipo	Funcional
Requisito	El sistema deberá mostrar un gráfico de consumo de las últimas 24 horas.		
Descripción	El dashboard muestra un gráfico de líneas con las últimas 48 lecturas de potencia (W) y voltaje (V), actualizadas cada 10 segundos mediante polling AJAX. Renderizado con Chart.js. Disponible únicamente en modo Premium.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	11	Tipo	Funcional
Requisito	El sistema deberá permitir exportar datos históricos de telemetría en formato CSV.		
Descripción	El administrador y operador en modo Premium pueden descargar un CSV con datos de telemetría PZEM y AHT10 filtrados por rango de fechas. El archivo incluye todos los campos de medición y el timestamp de cada lectura.		
Prioridad	Alta/Esencial ☐ Media/Deseado ☒ Baja/Opcional ☐		

Nro. de requisito	12	Tipo	Funcional
Requisito	El sistema deberá implementar rate limiting para proteger el endpoint de login.		
Descripción	Tras 5 intentos fallidos desde una misma IP en 10 minutos, el sistema bloquea esa IP por 15 minutos. El bloqueo se almacena en la tabla login_attempts, no en sesión, para evitar evasión por eliminación de cookies.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	13	Tipo	Funcional
Requisito	El sistema deberá registrar todos los eventos de seguridad relevantes.		
Descripción	La tabla security_logs registra: login_exitoso, login_fallido, bloqueo_ip, csrf_invalido y acceso_denegado. Cada registro incluye evento, usuario (si aplica), IP, user agent y timestamp.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	14	Tipo	Funcional
Requisito	El sistema deberá proveer una interfaz web local embebida en el ESP32.		
Descripción	El firmware sirve una interfaz HTTP en el puerto 80 de la IP local del ESP32, con autenticación. Permite visualizar y controlar las 4 tomas desde la LAN, sin necesidad de internet ni del portal web en la nube.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Nro. de requisito	15	Tipo	Funcional
Requisito	El sistema deberá enviar notificaciones por correo electrónico ante eventos críticos.		
Descripción	Se envían correos mediante PHPMailer + Gmail SMTP en los siguientes casos: alerta de umbral detectada, licencia próxima a vencer y recupero de contraseña. Los correos incluyen detalle del evento, PDU afectado y fecha/hora.		
Prioridad	Alta/Esencial ☒ Media/Deseado ☐ Baja/Opcional ☐		

Resumen de prioridades — 4.2

Nro.	Requisito (resumen)	Prioridad
1	El sistema deberá permitir encender y apagar individualmente cada toma...	Alta / Esencial
2	El sistema deberá publicar telemetría eléctrica en tiempo real cada 30...	Alta / Esencial
3	El sistema deberá publicar telemetría de temperatura y humedad cada 60...	Alta / Esencial
4	El sistema deberá detectar condiciones fuera de umbral y generar alert...	Alta / Esencial
5	El sistema deberá implementar autenticación con tres roles diferenciad...	Alta / Esencial
6	El sistema deberá permitir la gestión de usuarios (alta, baja, modific...	Alta / Esencial
7	El sistema deberá permitir la gestión de PDUs (alta, baja, modificació...	Alta / Esencial
8	El sistema deberá implementar un sistema de licencias Normal y Premium...	Alta / Esencial
9	El sistema deberá almacenar lecturas en un buffer offline cuando no ha...	Alta / Esencial
10	El sistema deberá mostrar un gráfico de consumo de las últimas 24 hora...	Alta / Esencial
11	El sistema deberá permitir exportar datos históricos de telemetría en ...	Media / Deseado
12	El sistema deberá implementar rate limiting para proteger el endpoint ...	Alta / Esencial
13	El sistema deberá registrar todos los eventos de seguridad relevantes.	Alta / Esencial
14	El sistema deberá proveer una interfaz web local embebida en el ESP32.	Alta / Esencial
15	El sistema deberá enviar notificaciones por correo electrónico ante ev...	Alta / Esencial

Interfaces de Software

Esta sección describe los componentes de software externos e internos con los que el sistema SmartRACK interactúa, tanto en el portal web como en el firmware del ESP32.

Backend — Portal web

Componente	Versión	Rol en el sistema
PHP	8.2+	Lenguaje principal del backend. Procesa todas las peticiones HTTP, maneja la sesión, la autenticación, los roles y la lógica de negocio.
MySQLi (extensión PHP)	8.2+	Interfaz de acceso a la base de datos MySQL. Se utiliza en modo procedural con <code>mysqli_real_escape_string()</code> para prevenir inyección SQL.
PHPMailer	6.x	Librería para el envío de correos electrónicos vía Gmail SMTP con TLS. Utilizada para alertas, avisos de licencia y recupero de contraseña.
phpdotenv	5.x	Carga variables de entorno desde el archivo <code>.env</code> (credenciales DB, MQTT, SMTP). Evita hardcodear credenciales en el código fuente.
php-mqtt/client	2.3.2	Cliente MQTT para PHP utilizado en <code>mqtt_worker.php</code> . Suscribe a todos los tópicos del broker y procesa telemetría, estados de toma, alertas y LWT.
Composer	2.x	Gestor de dependencias PHP. Administra PHPMailer, phpdotenv y php-mqtt/client.

Frontend — Portal web

Componente	Versión	Rol en el sistema
Tabler.io	1.0.0-beta20	Framework de UI basado en Bootstrap 5. Provee el layout del dashboard, componentes visuales, cards, tablas y navegación.
Bootstrap	5 (incluido en Tabler)	Sistema de grilla y componentes base del portal web.
Chart.js	latest (CDN)	Librería de gráficos JavaScript. Renderiza el gráfico de líneas de potencia y voltaje en los dashboards Premium.
Font Awesome	5.15.4	Librería de íconos vectoriales utilizada en el sidebar, botones, métricas y badges del portal.
Montserrat	Google Fonts	Tipografía principal del portal web, coherente con la identidad de marca de AucaTek.
Vanilla JS	ES6+	JavaScript sin frameworks. Maneja el polling AJAX cada 10 segundos, el control de tomas, el modal de verificación de identidad y la navegación activa del sidebar.

Firmware — ESP32

Librería	Fuente	Rol en el firmware
Arduino core para ESP32	Espressif (Arduino IDE)	Core base del firmware. Provee el entorno de desarrollo, acceso a periféricos y funciones del sistema.
Ethernet.h	Arduino (built-in)	Control del chip W5500 vía SPI. Maneja la conexión Ethernet, DHCP y el stack TCP/IP en hardware.
SSLClient	OPeN Lab — Noah Koontz	Implementa TLS sobre cualquier clase Client de Arduino. Envuelve EthernetClient para proveer comunicación MQTT cifrada con BearSSL.
PubSubClient	Knolleary	Cliente MQTT para Arduino. Maneja la conexión, suscripción a tópicos, publicación de mensajes y el loop de mensajes entrantes.
PZEM004Tv30	dbuezas	Librería de comunicación con el sensor PZEM-004T v3 por UART. Provee métodos para leer voltaje, corriente, potencia, PF, frecuencia y energía.
AHTxx	enjoyneering	Librería de comunicación con el sensor AHT10 por I2C. Provee métodos para leer temperatura y humedad.
ArduinoJson	6.x	Serialización y deserialización de JSON en el firmware. Usado para construir payloads MQTT y parsear comandos entrantes.
NTPClient	Fabrice Weinberg	Cliente NTP para sincronización horaria. Se conecta a pool.ntp.org con offset UTC-3 (Argentina).
EthernetUdp	Arduino (built-in)	Soporte UDP para el cliente NTP sobre Ethernet.

Interfaces de Comunicación

Esta sección describe los protocolos y canales de comunicación utilizados en SmartRACK para el intercambio de datos entre el firmware, el backend y los servicios externos.

MQTT sobre TLS — Firmware ↔ Backend

La comunicación principal entre el firmware del ESP32 y el backend PHP se realiza mediante el protocolo MQTT v3.1.1 sobre TLS (puerto 8883) a través del broker HiveMQ Cloud. Esta comunicación es bidireccional: el firmware publica telemetría y alertas, y el backend publica comandos de control de tomas.

Tópico	Dirección	Descripción
pdu/{codigo}/telemetria/pzem	ESP32 → Backend	Telemetría eléctrica cada 30s: voltage_v, current_a, power_w, power_factor, frequency_hz, energy_kwh, timestamp.
pdu/{codigo}/telemetria/aht10	ESP32 → Backend	Telemetría ambiental cada 60s: temperature_c, humidity_pct, timestamp.
pdu/{codigo}/comando/toma/{N}	Backend → ESP32	Comando de control de toma N (1-4). Payload JSON: {"command": "on"} o {"command": "off"}.
pdu/{codigo}/estado/toma/{N}	ESP32 → Backend	Confirmación del nuevo estado de la toma N tras ejecutar un comando. Payload JSON: {"state": 1, "timestamp": "..."}.
pdu/{codigo}/alertas	ESP32 → Backend	Alerta de umbral detectada por el firmware. Payload JSON: {"tipo": "...", "valor": X, "umbral": Y, "timestamp": "..."}.
pdu/{codigo}/lwt	Broker → Backend	Last Will Testament. Publicado automáticamente por el broker si el ESP32 se desconecta abruptamente. Payload: {"status": "offline", "timestamp": "..."}.

HTTP/HTTPS — Cliente ↔ Portal web

La comunicación entre el navegador del cliente y el portal web se realiza mediante HTTP/HTTPS. En producción (alwaysdata) el acceso es exclusivamente por HTTPS con certificado válido. El portal implementa una arquitectura de servidor tradicional (PHP renderiza HTML) combinada con llamadas AJAX para actualización en tiempo real.

Endpoint	Método	Descripción
index.php	GET / POST	Página de login. POST procesa las credenciales y establece la sesión.
dashboard_admin.php	GET	Dashboard del Administrador. Requiere rol admin y sesión activa.
dashboard_operator.php	GET	Dashboard del Operador. Requiere rol operator.

Endpoint	Método	Descripción
dashboard_viewer.php	GET	Dashboard de solo lectura. Requiere rol viewer.
toggle_outlet.php	POST	Cambia el estado de una toma. Requiere CSRF token válido. Publica comando MQTT al ESP32.
get_telemetry.php	GET	Devuelve JSON con telemetría, estado de tomas y alertas activas. Usado por el polling AJAX cada 10 segundos.
get_logs.php	GET	Devuelve JSON con los últimos 10 eventos del log del sistema.
resolver_alerta.php	POST	Marca una alerta como resuelta. Requiere CSRF token.
exportar_datos.php	POST	Genera y descarga un CSV con telemetría filtrada por fechas. Solo Premium.
verify_password.php	POST	Verifica la contraseña del usuario para operaciones sensibles (gestión de usuarios).
admin_usuarios.php	GET / POST	CRUD de usuarios. Solo Administrador. Requiere verificación previa de identidad.
admin_pdus.php	GET / POST	CRUD de PDUs. Solo Administrador.
cerrar_sesion.php	GET	Destruye la sesión activa y redirige al login.

UART — ESP32 ↔ PZEM-004T

La comunicación con el sensor PZEM-004T v3 se realiza mediante UART (RS-485 simplificado) sobre el puerto Serial2 del ESP32 (RX2=GPIO16, TX2=GPIO17) a 9600 baudios. La librería PZEM004Tv30 abstrae el protocolo Modbus RTU utilizado por el sensor. El ESP32 actúa como maestro y el PZEM como esclavo — el ESP32 solicita una lectura y el PZEM responde con todos los parámetros eléctricos en un único frame.

I2C — ESP32 ↔ AHT10

La comunicación con el sensor AHT10 se realiza mediante el protocolo I2C a 400kHz (Fast Mode) sobre los pines SDA=GPIO21 y SCL=GPIO22 del ESP32. La dirección I2C del AHT10 es 0x38. La librería AHTxx abstrae la inicialización del sensor, el disparo de medición y la lectura de los registros de temperatura y humedad.

SPI — ESP32 ↔ W5500

La comunicación con el módulo Ethernet W5500 (USR-ES1) se realiza mediante SPI en el bus VSPI del ESP32: SCK=GPIO18, MISO=GPIO19, MOSI=GPIO23, CS=GPIO5, RESET=GPIO4. El W5500 opera por polling — el pin INT queda sin conectar y no se usa el modo de interrupciones. La librería Ethernet.h maneja toda la comunicación con el chip y expone una interfaz Client compatible con SSLClient y PubSubClient.

SMTP — Backend ↔ Gmail

El envío de correos electrónicos se realiza mediante PHPMailer conectándose al servidor SMTP de Gmail (smtp.gmail.com) en el puerto 587 con autenticación STARTTLS. La cuenta de correo es ezequelmorgado07@gmail.com con contraseña de aplicación generada desde la consola de Google. Esta configuración se almacena en el archivo .env del servidor y nunca se hardcodea en el código fuente.

Nota: La cuenta de correo utilizada actualmente (ezequelmorgado07@gmail.com) es provisional y corresponde al entorno de desarrollo del proyecto. En la versión comercial del producto, AucaTek reemplazará esta cuenta por una dirección oficial (por ejemplo, notificaciones@aucatek.com.ar) utilizando un servicio de correo transaccional como SendGrid o Mailgun, que están diseñados específicamente para el envío masivo y confiable de notificaciones automáticas. El destinatario de cada correo ya es el mail del usuario registrado en el portal — cada cliente recibe las alertas en su propia cuenta, no en la del equipo de desarrollo.

NTP — Firmware ↔ Internet

El firmware sincroniza la hora del ESP32 al arrancar mediante el protocolo NTP (UDP, puerto 123) contra el servidor pool.ntp.org. Se aplica un offset de UTC-3 correspondiente a la zona horaria de Argentina (ART). La sincronización se renueva automáticamente cada 60 segundos durante la operación normal. El timestamp resultante se incluye en todos los mensajes MQTT publicados por el firmware.

Fin de la Sección 4 — Requisitos Específicos (4.1, 4.4 y 4.5)

5. Oferta de Gestión

5.2 Infraestructura Administrativa

El equipo de desarrollo de SmartRACK es un grupo interdisciplinario conformado por estudiantes de séptimo año del Instituto Leonardo Murialdo, organizado en torno al proyecto bajo la supervisión académica de los profesores Diego Martini, Lourdes Pedaci, Maximiliano Ariza y Gisela Bianchi, y con AucaTek como empresa cliente. La estructura del equipo sigue un modelo de organización matricial orientado al proyecto, donde cada integrante aporta desde su área de especialización.

Organización del equipo

El siguiente esquema representa la estructura del equipo del proyecto SmartRACK y sus áreas de trabajo:



Integrantes y responsabilidades

Nombre	Área	Herramientas principales	Responsabilidades
Ezequiel Morgado	Project Manager Backend Developer	XAMPP, VS Code, phpMyAdmin, alwaysdata, GitHub, Claude	Desarrollo del portal web, base de datos MySQL, integración MQTT, ciberseguridad, deploy en producción, API, integración general, control y gestión completa del proyecto
Camilo Spinelli	Hardware y Firmware	Arduino IDE, GitHub, Claude	Firmware ESP32 y sensores
Sofía Hermosilla	Frontend	VS Code, Chrome DevTools, Lighthouse, GitHub, Claude	Diseño e implementación de la interfaz web, experiencia de usuario, dashboards por rol, bocetos en papel.
Violeta Martini	Diseño Industrial	Fusion 360, AutoCAD	Diseño del gabinete PDU v1/2 y prototipo físico del dispositivo.

Nombre	Área	Herramientas principales	Responsabilidades
Ignacio Marticorena	Multimedia	CapCut, Runway ML, Kling AI, Adobe Illustrator	Logo del proyecto, identidad visual, video promocional, Marketing del producto.
Federico Sparta	Electrónica	Arduino IDE, KiCad, Claude	Realización de la placa en PCB para la v1 y v2, desarrollo del código para sensores y ESP32 con sus respectivas conexiones
Santiago Villarosa	Electrónica	Arduino IDE, KiCad, Claude	Realización de la placa en PCB para la v1 y v2, desarrollo del código para sensores y ESP32 con sus respectivas conexiones

Comunicación y coordinación

Herramienta	Uso en el proyecto
WhatsApp	Canal principal de comunicación diaria entre los integrantes del equipo. Se usa para coordinación rápida, consultas y avisos urgentes.
Google Sheets	Gestión de tareas, cronograma del proyecto (GANTT) y seguimiento del estado de cada actividad por área.
GitHub	Control de versiones del código fuente. Todos los integrantes del área de software son contribuidores del repositorio. Se utiliza la convención de commits convencionales (feat:, fix:, docs:).

Desarrollo de software

Herramienta	Uso en el proyecto
VS Code	Editor de código principal para el desarrollo del portal web (PHP, HTML, CSS, JavaScript) y edición del firmware.
XAMPP	Entorno de desarrollo local en Windows. Incluye Apache y MySQL para probar el portal web antes de deployar a producción.
phpMyAdmin	Gestión visual de la base de datos MySQL, tanto en el entorno local (XAMPP) como en producción (alwaysdata).
Arduino IDE	Entorno de desarrollo del firmware del ESP32. Compilación, carga del binario al microcontrolador y monitor serie para diagnóstico.
alwaysdata	Plataforma de hosting en producción. Aloja el portal web (PHP 8.2), la base de datos MySQL, el mqtt_worker.php como servicio y los archivos del proyecto.
Chrome DevTools	Herramienta de depuración del frontend. Inspección de elementos, consola de errores JavaScript, análisis de peticiones AJAX y validación de headers HTTP.

Herramientas utilizadas por categoría

Herramienta	Uso en el proyecto
Lighthouse	Herramienta de auditoría de calidad web integrada en Chrome DevTools. Se utilizó para medir y mejorar Performance (96), Accessibility (82) y Best Practices (100) del portal.

Diseño industrial y hardware

Herramienta	Uso en el proyecto
Fusion 360	Diseño 3D del gabinete PDU para la versión 2 del producto. A cargo de Violeta Martini. También utilizado por Martín Barral para el prototipo físico v1.
AutoCAD	Diseño de planos técnicos y esquemas eléctricos del PCB y del gabinete. Complementa a Fusion 360 para la documentación técnica del hardware.

Control de calidad y seguridad

Herramienta / Práctica	Uso en el proyecto
Pruebas funcionales manuales	Plan de pruebas ejecutado sobre el hardware real con el ESP32 conectado. Cubre control de tomas, telemetría MQTT, alertas, roles y exportación CSV.
phpMyAdmin	Revisión y validación de la estructura de la base de datos, consultas SQL y datos de producción.
GitHub (historial de commits)	Trazabilidad completa de los cambios en el código. El historial de commits es auditable y refleja el avance del proyecto semana a semana.

